F3: Engine Logic & Technical Whitepaper

By Basel Shokha

Problem Definition

Input:
• A weighted directed graph $G = (V, E)$ representing the regional map with a non-negative weight function w: $E \to \mathbb{R}^+$ representing road distances.
• A start node $s \in V$ and a destination node $t \in V$.
• A set of battery pile locations $B \subseteq V$, where each location $b_i \in B$ contains a discrete number of batteries defined by a capacity function $c_B : B \to \mathbb{N}$.
• A maximum operational battery capacity (range constraint) $X \in \mathbb{R}^+$.

Output:
The absolute maximum swarm throughput (integral flow) from s to t under the strict range constraint, alongside a mathematically rigorous identification of the structural network bottlenecks restricting further collection.

Algorithm Description

1. Execute Dijkstra's algorithm on G from the source s and from every single battery pile $b_i \in B$. Construct a condensed directed graph $G' = (V', E')$ where the viable hubs are $V' = \{s, t\} \cup B$. For any two nodes $u, v \in V'$, add a directed edge (u, v) to E' if their lightest path distance $d_u(v) \leq X$. Cache the physical sequence of nodes that induced this distance.

2. Construct the terminal Max-Flow network $N = (V_N, E_N)$. Apply the node-splitting technique to enforce the finite number of batteries at each pile. For every battery node $b \in B$, create two nodes b_{in} and b_{out} in V_N connected by a directed internal edge (b_{in}, b_{out}) with a capacity equal to the number of batteries at that pile, $c_B(b)$. The global source s and sink t are added to V_N without splitting.
Map the condensed edges from E' into N with capacity ∞ as follows:
• For an edge (s, b), add an edge $s \to b_{in}$.
• For an edge (b_i, b_j), add an edge $(b_i)_{out} \to (b_j)_{in}$.
• For an edge (b, t), add an edge $b_{out} \to t$.

3. Execute the Edmonds-Karp algorithm on N to push maximum integral flow from s to t. The algorithm repeatedly searches for augmenting paths using Breadth-First Search (BFS).

4. After computing the maximum flow, extract the resulting integral flow decomposition. Repeatedly start from s and follow edges carrying positive flow until reaching t, subtracting one unit of flow from each used edge. This produces a collection of $s \to t$ swarm paths. Then map each condensed network edge back to its corresponding physical road path in G.

5. Return the maximum flow value alongside the isolated bottleneck edges.

Proof of Correctness

Lemma: The node splitting transformation strictly enforces battery availability.

Proof: In a standard flow network, capacities restrict edges, not nodes. By splitting a battery pile b into b_{in} and b_{out} and routing all incoming pathways to b_{in} and all outgoing pathways from b_{out}, any physical unit traversing b is forcefully bottlenecked by the internal edge (b_{in}, b_{out}). Setting this edge's capacity to $c_B(b)$ and assigning ∞ capacity to the connecting road edges rigorously isolates the finite number of batteries as the sole limiting factor of the flow, accurately simulating that each passing unit consumes one battery. ■

Claim: *(the infinity case is trivial)*

We can safely swarm *K* B.A.S.E.L. units ⇔ a flow of size *K* exists in *N*.

(⇒) Proof of Safe Swarm to Flow: Suppose we have a valid, safe deployment of K individual units from s to t. Each unit traverses a physical path in G such that no subpath between battery piles exceeds X. These subpaths directly map to valid edges in G' (since they satisfy distance $\leq X$) and therefore to infinite-capacity edges in N. For every unit that stops at a battery pile b, we route (increase) 1 unit of flow through the internal edge (b_{in}, b_{out}). Because a safe physical swarm cannot consume more batteries than actually exist, the total flow accumulating through any (b_{in}, b_{out}) is mathematically guaranteed to be $\leq c_B(b)$, strictly respecting the network's capacity rules. Flow conservation is perfectly maintained as each unit physically enters and leaves a pile. Thus, the real-world paths form a legal flow of exactly size K in N.

(⇐) Proof of Flow to Safe Swarm: Suppose there is an integral flow of size K in N. By the flow decomposition theorem, this flow can be broken down into exactly K distinct $s \rightarrow t$ paths. Due to the strict capacities in N, at most $c_B(b)$ abstract flow paths pass through any internal edge (b_{in}, b_{out}), ensuring we don't over-deploy units to any battery pile. We then replace each abstract network edge with its cached physical subpath from G. Because every connection in N originated directly from G', its underlying physical distance is mathematically guaranteed to be $\leq X$. Therefore, mapping these K abstract flow paths back to G yields K valid, completely range-feasible routes. This guarantees the swarm is safe. ■

Time Complexity Analysis

Step 1 : Running Dijkstra's algorithm on G from s and from every battery pile $b \in B$ using a minimum heap requires $|B| + 1$ independent executions.

This takes exactly $O(|B| \cdot |E| \log |V|)$.

Step 2 : Building the flow network N constructs $|V_N| = 2|B| + 2$ nodes and at most $O(|B|^2)$ edges. This strict construction takes time bounded by $O(|B|^2)$.

Step 3 : The Edmonds-Karp algorithm performs at most $O(|V_N||E_N|)$ flow augmentations. Each BFS traversal takes $O(|E_N|)$. The time complexity of this phase evaluates to $O(|V_N||E_N|^2)$. Substituting our bounded network size yields $O(|B| \cdot (|B|^2)^2) = O(|B|^5)$.

Step 4 : After max flow is computed, decompose the integral flow into individual $s \to t$ paths. Repeatedly start from s and follow edges carrying positive flow until reaching t, decrementing one unit of flow along the path. For each extracted path, map condensed edges back to their corresponding physical paths in G.

This decomposition step takes $O(|f^*| \cdot |E_N|)$ time, where $|f^*|$ is the value of the maximum flow. Since $|E_N| = O(|B|^2)$, Step 4 runs in $O(|f^*| \cdot |B|^2)$.

Total Time Complexity:
Combining the initial routing, transformation, flow maximization, and flow decomposition yields:

$$\mathbf{O(|B| \cdot |E| \log |V| + |B|^5 + |f^*| \cdot |B|^2)}$$

In a real-world mapping scenario, the sheer density of roads and intersections astronomically outscales the number of discrete battery piles ($|E|, |V| \gg |B|$). Furthermore, assuming the total achievable flow f^* remains relatively small compared to $|B|^3$ (i.e., the swarm size is not excessively large), the flow decomposition term $O(|f^*| \cdot |B|^2)$ does not dominate the overall runtime.

Under these practical conditions, the $|B|^5$ and $|f^*| \cdot |B|^2$ terms behave as near-constant factors, and the overall engine performance is effectively dominated by the initial shortest-path computations, yielding an optimized runtime of:

$$\mathbf{O(|B| \cdot |E| \log |V|)}$$